Below are **simple, exam-ready Java programs** for all the questions. You can copy each program separately into a `.java` file.

---

1. Abstract Class – Shape, Rectangle, Triangle and Circle

Java

```
abstract class Shape {


    abstract void printArea();
}

class Rectangle extends Shape {
    Rectangle(int length, int breadth) {
        a = length;
        b = breadth;
    }

    void printArea() {
        System.out.println("Area of Rectangle = " + (a * b));
    }
}

class Triangle extends Shape {
    Triangle(int base, int height) {
        a = base;
        b = height;
    }

    void printArea() {
        System.out.println("Area of Triangle = " + (0.5 * a * b));
    }
}

class Circle extends Shape {
    Circle(int radius) {
        a = radius;
    }

    void printArea() {
        System.out.println("Area of Circle = " + (Math.PI * a * a));
    }
}

class ShapeDemo {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(10, 5);
        Triangle t = new Triangle(10, 5);
```

```
        Circle c = new Circle(7);

        r.printArea();
        t.printArea();
        c.printArea();
    }
}
```

---

2. Multilevel Inheritance

Java

```
class GrandFather {
    void showGrandFather() {
        System.out.println("Grandfather class");
    }
}

class Father extends GrandFather {
    void showFather() {
        System.out.println("Father class");
    }
}

class Son extends Father {
    void showSon() {
        System.out.println("Son class");
    }
}

class MultilevelDemo {
    public static void main(String[] args) {
        Son s = new Son();

        s.showGrandFather();
        s.showFather();
        s.showSon();
    }
}
```

---

3. Final Keyword

Java

```
class FinalDemo {
    final int VALUE = 100;
```

```
    final void display() {
        System.out.println("This is a final method.");
        System.out.println("Value = " + VALUE);
    }

    public static void main(String[] args) {
        FinalDemo obj = new FinalDemo();
        obj.display();
    }
}
```

Note: A `final` variable cannot be changed, and a `final` method cannot be overridden.

---

4. StringBuffer Mutability – Palindrome

Java

```
import java.util.Scanner;

class Palindrome {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        StringBuffer sb = new StringBuffer(str);
        String reverse = sb.reverse().toString();

        if (str.equals(reverse))
            System.out.println("The string is a palindrome.");
        else
            System.out.println("The string is not a palindrome.");
    }
}
```

---

5. Abstract Class and Abstract Methods

Java

```
abstract class Animal {
    abstract void sound();

    void eat() {
        System.out.println("Animal eats food");
```

```
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

class AbstractDemo {
    public static void main(String[] args) {
        Dog d = new Dog();

        d.sound();
        d.eat();
    }
}
```

---

6. Super Keyword

Java

```
class Parent {
    int number = 100;

    void display() {
        System.out.println("Parent class method");
    }
}

class Child extends Parent {
    int number = 200;

    void display() {
        System.out.println("Child class method");
    }

    void show() {
        System.out.println("Parent variable = " + super.number);
        super.display();
        System.out.println("Child variable = " + number);
    }
}

class SuperDemo {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();
    }
```

```
}
```

---

7. User-Defined Package – `sample`

File 1: `sample/Test.java`

Java

```java
package sample;

public class Test {
    public void display() {
        System.out.println("This is a user-defined package.");
    }
}
```

File 2: `PackageDemo.java`

Java

```java
import sample.Test;

class PackageDemo {
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display();
    }
}
```

Compile and Run

```
javac -d . Test.java
javac PackageDemo.java
java PackageDemo
```

---

8. Interface

Java

```java
interface Animal {
    void sound();
}
```

```java
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}

class InterfaceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

---

9. User-Defined Exception Using `throw`

Java

```java
class AgeException extends Exception {
    AgeException(String message) {
        super(message);
    }
}

class UserExceptionDemo {
    public static void main(String[] args) {
        int age = 15;

        try {
            if (age < 18)
                throw new AgeException("Age must be 18 or above");

            System.out.println("Eligible");
        } catch (AgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

---

10. Multiple Inheritance Using Interface

Java does not support multiple inheritance using classes, but it supports it through interfaces.

Java

```java
interface Father {
```

```java
    void showFather();
}

interface Mother {
    void showMother();
}

class Child implements Father, Mother {
    public void showFather() {
        System.out.println("Father interface");
    }

    public void showMother() {
        System.out.println("Mother interface");
    }
}

class MultipleInheritanceDemo {
    public static void main(String[] args) {
        Child c = new Child();

        c.showFather();
        c.showMother();
    }
}
```

---

11. Hierarchical Inheritance

Java

```java
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows");
    }
}

class HierarchicalDemo {
```

```
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();

        Cat c = new Cat();
        c.eat();
        c.meow();
    }
}
```

---

12. Exception Handling Using `throws`

Java

```
class ThrowsDemo {

    static void divide() throws ArithmeticException {
        int a = 10;
        int b = 0;

        System.out.println(a / b);
    }

    public static void main(String[] args) {
        try {
            divide();
        } catch (ArithmeticException e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

---

13. Single Inheritance

Java

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
```

```
        System.out.println("Dog barks");
    }
}

class SingleInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();

        d.eat();
        d.bark();
    }
}
```

---

14. Exception Handling – Multiple Catch Blocks

Java

```
class MultipleCatchDemo {
    public static void main(String[] args) {

        try {
            int a[] = {10, 20, 30};

            System.out.println(a[5]);

            int x = 10 / 0;
            System.out.println(x);
        }

        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index is out of bounds.");
        }

        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }

        catch (Exception e) {
            System.out.println("Some other exception occurred.");
        }
    }
}
```

---

15. Armstrong Number Using `if-else` and `for` Loop

Java

```java
import java.util.Scanner;

class Armstrong {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        int original = n;
        int sum = 0;

        for (; n != 0; n = n / 10) {
            int digit = n % 10;
            sum = sum + digit * digit * digit;
        }

        if (sum == original)
            System.out.println(original + " is an Armstrong number.");
        else
            System.out.println(original + " is not an Armstrong number.");
    }
}
```

Example: `153 = 1³ + 5³ + 3³ = 153`

---

16. Vehicle Interface

Java

```java
interface Vehicle {
    void start();
    void stop();
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }

    public void stop() {
        System.out.println("Car stops");
    }
}

class Bike implements Vehicle {
```

```
    public void start() {
        System.out.println("Bike starts");
    }

    public void stop() {
        System.out.println("Bike stops");
    }
}

class VehicleDemo {
    public static void main(String[] args) {
        Car c = new Car();
        c.start();
        c.stop();

        Bike b = new Bike();
        b.start();
        b.stop();
    }
}
```

---

17. Exception Handling Using `try`, `catch` and `finally`

Java

```
class FinallyDemo {
    public static void main(String[] args) {

        try {
            int a = 10;
            int b = 0;

            System.out.println(a / b);
        }

        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }

        finally {
            System.out.println("Finally block is always executed.");
        }
    }
}
```

Quick exam revision

Topic	Main keyword/concept
Abstract class	`abstract`
Single inheritance	`extends`
Multilevel inheritance	Multiple `extends` levels
Hierarchical inheritance	One parent, multiple children
Multiple inheritance	`interface`
Interface	`implements`
Super keyword	`super`
Final keyword	`final`
User-defined exception	`throw`
Exception declaration	`throws`
Exception handling	`try-catch`
Always-executed block	`finally`
StringBuffer	Mutable string
Package	`package`, `import`

Files, images, and data analysis are unavailable until usage resets tomorrow at 2:24 AM.
Continue chatting with text only, or upgrade for more access.

Try Plus free